48. SELinux/AppArmor Awareness

48. SELinux/AppArmor Awareness

Phase: Phase 9 — Security and SSH
Estimated time: 1–2 days
Slug: `48-selinux-apparmor`

Learning objectives

- Know which MAC system your distro uses
- Read basic denials from logs
- Avoid setenforce 0 as permanent fix
- Apply targeted policy changes properly

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Classic Unix permissions (user, group, other) answer "who owns this file". They do not answer "should a web server ever read `/home` or open a shell". Mandatory Access Control (MAC) adds a second, policy-driven layer that even root-owned processes cannot bypass. When a web app is compromised, MAC is often what stops the attacker at one directory instead of the whole host.

It also causes a famous class of mystery failures: permissions look right, the service is running, yet "Permission denied" appears. If you do not know MAC exists, you lose hours. If you answer with `setenforce 0` and move on, you lose the protection.

Which one you meet depends on the distro family: RHEL, Fedora, Rocky, and Alma use SELinux; Ubuntu, Debian-family, and SUSE use AppArmor (SUSE moved to SELinux in recent versions, so check).

Core concepts

Item	SELinux	AppArmor
Typical distros	RHEL, Fedora, Alma	Ubuntu, Debian
Model	labels on files and processes	path-based profiles
Status	`getenforce`, `sestatus`	`aa-status`
Modes	enforcing, permissive, disabled	enforce, complain
Denial log	`ausearch -m AVC`, audit.log	`journalctl`, `dmesg`
Fix files	`restorecon`, `semanage fcontext`	edit profile, `aa-logprof`
Fix toggles	`setsebool -P`	profile abstractions

```
process (label httpd_t) --wants--> file (label user_home_t)
              |                          |
              +----> kernel policy check --> ALLOW / DENY
                        (after normal chmod/chown checks)
deny -> audit record (AVC) -> you read it, fix label or policy,
        NOT disable the whole system
```

Rule: chmod passes first, then MAC is consulted. Both must allow.

Worked examples

Example 1: which system is this host using?

```
getenforce 2>/dev/null || echo "no selinux"
sudo aa-status 2>/dev/null | head -3 || echo "no apparmor"
cat /etc/os-release | grep -E '^(ID|ID_LIKE)='
```

Example 2: SELinux, web server cannot read content from a custom path.

```
ls -Z /srv/site/index.html
# unconfined_u:object_r:var_t:s0   <- wrong type for web content
sudo semanage fcontext -a -t httpd_sys_content_t "/srv/site(/.*)?"
sudo restorecon -Rv /srv/site
```

Example 3: SELinux, read the denial and use a boolean.

```
sudo ausearch -m AVC -ts recent | tail -5
sudo setsebool -P httpd_can_network_connect on
```

Example 4: AppArmor, find and loosen one profile temporarily.

```
sudo aa-status
sudo journalctl -k | grep -i apparmor | tail
sudo aa-complain /usr/sbin/mysqld
```

```
# test, collect denials, update the profile, then:
sudo aa-enforce /usr/sbin/mysqld
```

Example 5: testing with permissive mode, honestly.

```
sudo setenforce 0     # temporary, lasts until reboot
# reproduce failure; if it vanishes, it IS a MAC denial
sudo setenforce 1     # put it back immediately
```

This is a diagnostic, not a fix.

Mini-lab

1. Identify the distro family with `/etc/os-release` and decide which MAC system you should expect.
2. Run `getenforce` or `aa-status`. Record the mode.
3. On SELinux hosts: `ls -Z /var/www /etc/passwd` and compare the type fields. On AppArmor hosts: list loaded profiles and count enforced ones.
4. Make a file in `/tmp/site` and compare `ls -Z` before and after `restorecon` (SELinux) or note its path (AppArmor).
5. Find where denials are logged on your host and trigger one safely, if you can, by running a confined binary against an unexpected path.
6. Write the three-line playbook: read denial, find smallest fix, re-test in enforcing mode.

Decision guide / failure modes

Symptom	Likely cause	Right action
Perms fine, access denied	MAC label or profile	read AVC or audit log
Moved web root	wrong file context	`semanage fcontext` + `restorecon`
App must connect outbound	SELinux boolean off	`setsebool -P name on`
Need new path for daemon	AppArmor profile lacks rule	edit profile, reload
“Just disable it”	pressure	use permissive per-domain, fix, re-enable

Failure modes: `setenforce 0` left in place and `SELINUX=disabled` in config; files copied with `cp` instead of `cp --preserve=context` or relabeled; `mv` keeping the old label; giving up on AppArmor and unloading all profiles. Fix the smallest thing, in enforcing mode.

Common mistakes

- setenforce 0 forever

- Disabling AppArmor globally
- Ignoring AVC denial logs

Exercises

Work in `exercises/48-selinux-apparmor/`. Read that folder's `README.md` fully before coding.

```
cd exercises/48-selinux-apparmor
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Open `exercise.sh` and implement `mac_for_distro`. It takes one argument, a distro family name, and prints the Mandatory Access Control system that family uses by default. Handle at least `rhel` and `ubuntu`. Output exactly one lowercase word per call: no spaces, no extra text. Think about the lesson's distro table and keep the function a simple case statement. After the tests pass, run the status command appropriate to your lab VM and record the mode in your notes. Also write one sentence on why `setenforce 0` is a diagnostic step and not a fix.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/48-selinux-apparmor/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. SELinux vs AppArmor typical distros?
2. Why not disable MAC?
3. Where see denials?

Further reading

- https://www.selinuxproject.org/page/Main_Page